

## 1. Long Running Process

**Concept :** A process is a running program. When **task1** runs linux creates a process for it and allocates resources. Sleep() keeps it alive for about 30 seconds, so ps can show its details. The & symbol runs it in the background , freeing the terminal for other commands.

Source code :

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("I am starting...\n");

    // Loop for 30 seconds
    for (int i = 1; i <= 30; i++) {
        sleep(1); // Pauses execution for 1 second
    }

    printf("I am finished.\n");

    return 0;
}
```

Process :

```
root@1cd23b9c6365:/home/os# I am starting...
I am finished.
```

```
root@1cd23b9c6365:/# ps aux | grep task1
```

```
root      22  0.3  0.0   2760  1408 pts/1    S   08:27   0:00 ./task1
root      24  7.6  0.0   3684  1920 pts/0    S+  08:27   0:00 grep --color=auto task1
```

```
root@1cd23b9c6365:/# ps aux | grep task1
```

```
root      26  0.0  0.0   3684  2048 pts/0    S+  08:28   0:00 grep --color=auto task1
root@1cd23b9c6365:/#
```

← starting

finished

## 2. Process Identity (PID and PPID)

### Objective:

To identify the process ID (PID) and Parent Process ID (PPID) of a running C program and verify the process information using linux OS commands.

### Concept:

A **Process ID (PID)** is a unique number assigned to every running process in Linux.

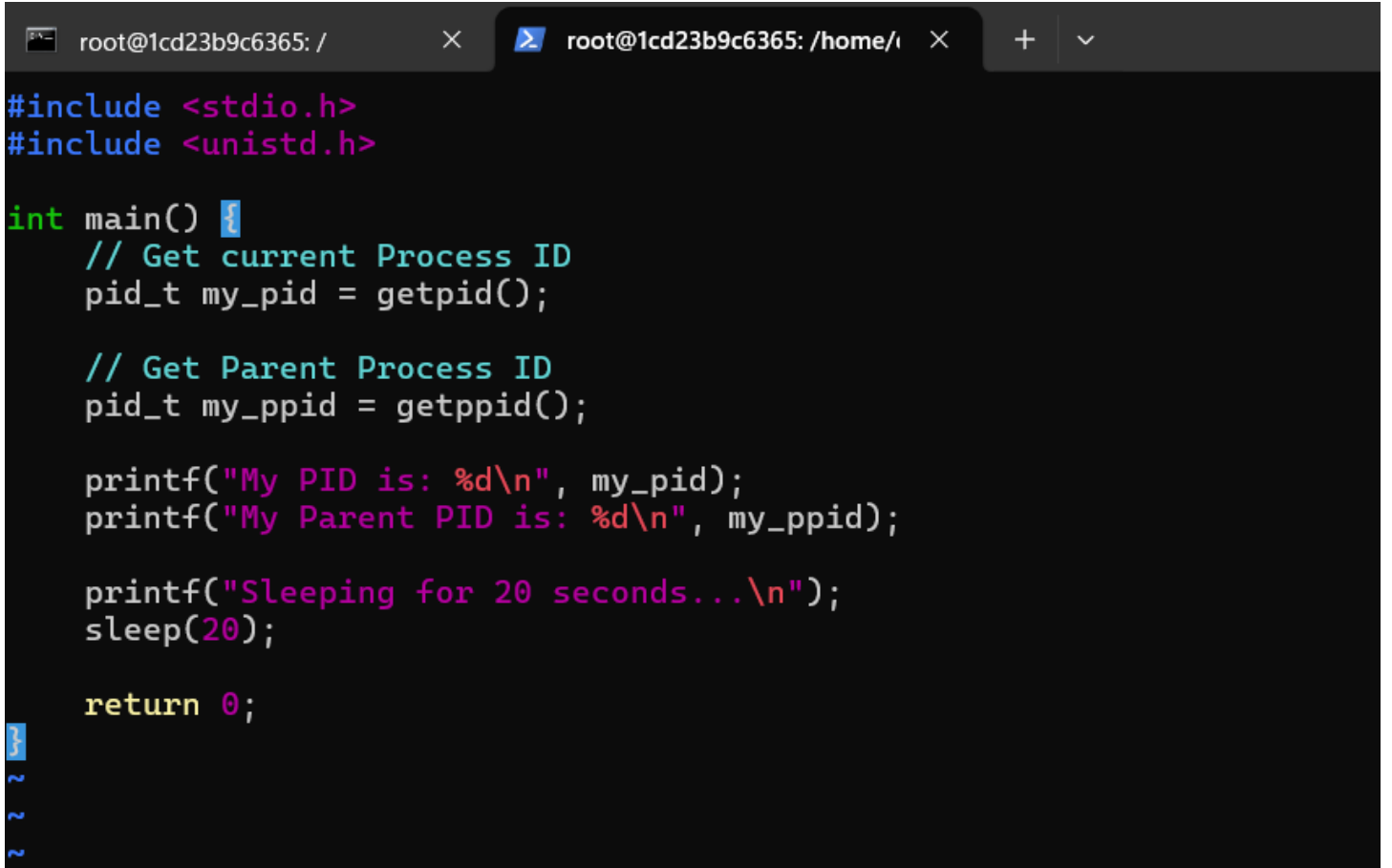
A **Parent Process ID (PPID)** identifies the process that created the current process.

The getpid() function returns the PID of the current process.

The getppid() function returns the PID of its parent process.

In Docker, a process may have **PPID 1**, depending on the container structure.

### Source code

A screenshot of a terminal window with two tabs. The first tab is titled 'root@1cd23b9c6365: /' and the second is 'root@1cd23b9c6365: /home/i'. The terminal displays C code that includes <stdio.h> and <unistd.h>. The main function calls getpid() to get the current process ID and getppid() to get the parent process ID. It then prints these values using printf and sleeps for 20 seconds before returning 0. The code is as follows:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    // Get current Process ID
    pid_t my_pid = getpid();

    // Get Parent Process ID
    pid_t my_ppid = getppid();

    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is: %d\n", my_ppid);

    printf("Sleeping for 20 seconds...\n");
    sleep(20);

    return 0;
}
```

## Compilation

```
root@1cd23b9c6365:/home/os# gcc -o task2 task2_identity.c
root@1cd23b9c6365:/home/os# ls
main      main.i    main.s    nano.c    nano.o    practice  practice1.i  task1_ps.c  task2.c
main.c    main.o    nano      nano.i    nano.s    practice1.c  task1       task2       task2_identity.c
root@1cd23b9c6365:/home/os#
```

## Process :

```
root@1cd23b9c6365:/home/os# ./task2 &
[2] 51
[1] Done
root@1cd23b9c6365:/home/os# ./task2
My PID is: 51
My Parent PID is: 9
Sleeping for 20 seconds...
root@1cd23b9c6365:/home/os#
```

```
root@1cd23b9c6365:/# ps -p 51 -o pid,ppid,cmd
PID    PPID  CMD
51      9     ./task2
root@1cd23b9c6365:/#
```

### 3. Exit Codes and OS feedback

#### Objective :

To understand how a C program communicates its execution status to the operating system using exit codes and to verify the returned status using the linux shell

#### Concept:

A program returns an **integer exit status** to indicate whether it executed successfully.

return 0 indicates **successful execution**, while a non-zero value indicates an **error or failure**.

In this task, positive input returns 0, while zero or negative input returns 1.

The Linux shell stores the exit status in \$?, which can be displayed using echo \$?.

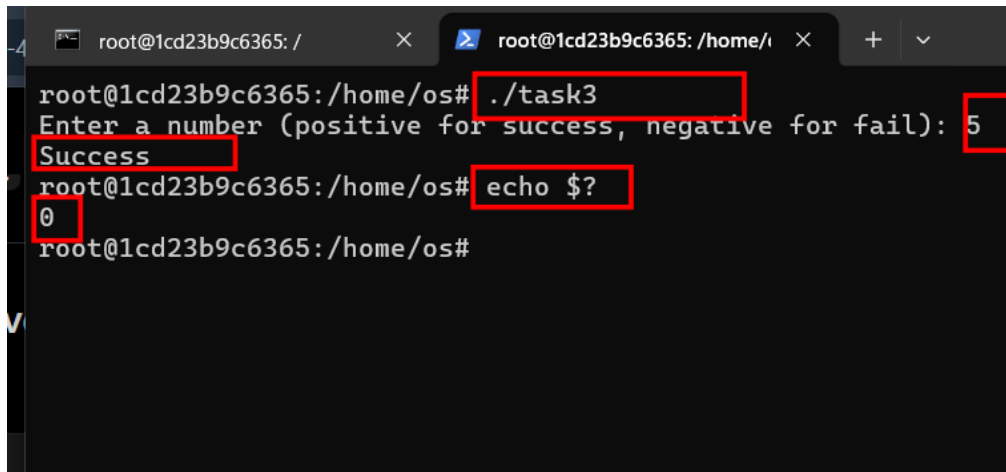
#### Source Code

```
root@1cd23b9c6365: /  
root@1cd23b9c6365: /home/i  
+ v  
#include <stdio.h>  
  
int main() {  
    int num;  
  
    printf("Enter a number (positive for success, negative for fail): ");  
    scanf("%d", &num);  
  
    if (num > 0) {  
        printf("Success\n");  
        return 0; // Tell OS: Success  
    } else {  
        printf("Failure\n");  
        return 1; // Tell OS: Error  
    }  
}
```

#### Compilation

```
root@1cd23b9c6365:/home/os# vim task3_exit.c  
[2]+  Done ./task2  
root@1cd23b9c6365:/home/os# gcc task3_exit.c -o task3  
root@1cd23b9c6365:/home/os# ls  
main      main.i    main.s    nano.c    nano.o    practice  practice1.i  task1_ps.c  task2.c    task3      task3_exit.c  
main.c    main.o    nano      nano.i    nano.s    practice1.c  task1       task2      task2_identity.c  
root@1cd23b9c6365:/home/os#
```

## Process :

A terminal window with a dark background. The prompt is 'root@1cd23b9c6365: /'. The user enters './task3'. The program prompts 'Enter a number (positive for success, negative for fail):'. The user enters '5'. The program outputs 'Success'. The user enters 'echo \$?'. The program outputs '0'.

```
root@1cd23b9c6365: /home/os# ./task3
Enter a number (positive for success, negative for fail): 5
Success
root@1cd23b9c6365: /home/os# echo $?
0
root@1cd23b9c6365: /home/os#
```

## 4. Standard I/O Streams

### Objective

To demonstrate how a C program uses standard input (stdin) to receive data from the user and standard output (stdout) to display information on the terminal.

### Concept

**Standard input (stdin)** is the default stream used to receive input, usually from the keyboard.

**Standard output (stdout)** is the default stream used to display output, usually on the terminal. In this task, scanf() reads the user's name from stdin, while printf() displays the greeting through stdout.

### Source Code:

```
root@1cd23b9c6365: / × root@1cd23b9c6365: /home/ × + v
#include <stdio.h>

int main() {
    char name[50];

    printf("Enter your name: ");

    // Read string from standard input
    scanf("%s", name);

    printf("Hello, %s! Welcome to OS Class.\n", name);

    return 0;
}
```

## Compilation

```
root@1cd23b9c6365:/home/os# ls
main      main.o  nano.c  nano.s      practice1.i  task2      task3
main.c    main.s  nano.i  practice    task1        task2.c    task3_exit.c
main.i    nano    nano.o  practice1.c task1_ps.c   task2_identity.c task4_input.c
root@1cd23b9c6365:/home/os# gcc task4_input.c -o task4
root@1cd23b9c6365:/home/os#
root@1cd23b9c6365:/home/os# ./task4
Enter your name: Aashish Dahal
Hello, Aashish! Welcome to OS Class.
root@1cd23b9c6365:/home/os#
```

## Result

```
root@1cd23b9c6365: /home/ × + v
root@1cd23b9c6365:/# ls
bin boot dev etc home lib lib64 media mnt opt os pr
root@1cd23b9c6365:/# cd home/os
root@1cd23b9c6365:/home/os# ls
main      main.o  nano.c  nano.s      practice1.i  task2      task3
main.c    main.s  nano.i  practice    task1        task2.c    task3_exit.c
main.i    nano    nano.o  practice1.c task1_ps.c   task2_identity.c task4_input.c
root@1cd23b9c6365:/home/os# gcc task4_input.c -o task4
root@1cd23b9c6365:/home/os#
root@1cd23b9c6365:/home/os# ./task4
Enter your name: Aashish Dahal
Hello, Aashish! Welcome to OS Class.
root@1cd23b9c6365:/home/os#
```

## 5. Conditional Execution and Termination

### Objective

To demonstrate how user input controls program execution and how a C program communicates its final execution status to the operating system through exit codes.

### Concept

This task demonstrates conditional execution and process termination in Linux. The program first uses `getpid()` to display its Process ID (PID), which uniquely identifies the running process. The user then enters a choice. If the choice is 1, the program continues, waits for 5 seconds using `sleep()`, and returns exit code 0, indicating successful execution. If the choice is 0, the program exits immediately with exit code 1, indicating an unsuccessful or aborted condition. The exit status can be checked in the Linux terminal using `echo $?`.

### Source Code:

```
root@1cd23b9c6365: /home/i  X + v
#include <stdio.h>
#include <unistd.h>

int main() {
    int choice;

    // Print PID at start
    printf("Current PID: %d\n", getpid());

    printf("Do you want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Continuing...\n");
        sleep(5);
        return 0; // Success
    } else {
        printf("Exiting...\n");
        return 1; // Failure/Abort
    }
}
```

## Compilation

```
root@1cd23b9c6365:/home/os# vim task5_control.c
root@1cd23b9c6365:/home/os#
root@1cd23b9c6365:/home/os# gcc task5_control.c -o task5
root@1cd23b9c6365:/home/os# ./task5
```

## Result

```
root@1cd23b9c6365:/home/os# ./task5 ①
Current PID: 32
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing...
root@1cd23b9c6365:/home/os# ./task5 ②
Current PID: 33
Do you want to continue? (1 for Yes, 0 for No): 0
Exiting...
root@1cd23b9c6365:/home/os#
root@1cd23b9c6365:/home/os#
```